

Simulazione drone in ambiente warehouse

Architettura, contenuti del repository, collegamento PX4-Gazebo-ROS 2, procedura di decollo e ottimizzazioni della simulazione.

3

servizi Docker Compose

71

istanze warehouse incluse

10 Hz

pubblicazione setpoint

1.0×

fattore tempo configurato

In breve

Il progetto integra PX4 SITL con Gazebo Harmonic e un mondo di magazzino composto da modelli locali. Un container ROS 2 Humble ospita Micro XRCE-DDS Agent e gli strumenti applicativi; il package `warehouse_bringup` fornisce un launch configurabile e un nodo di esempio che richiede la modalità Offboard, arma il drone e mantiene un setpoint a un metro sopra la posizione iniziale.

PX4 SITL

Autopilota e modello
`x500_depth`

Gazebo Harmonic

Fisica, sensori e mondo
warehouse

XRCE-DDS Agent

Trasporto PX4 ↔ ROS 2 su
UDP 8888

ROS 2 Humble

Bridge, bringup e
comando Offboard

Nota sulle prestazioni. La configurazione attuale non imposta una simulazione più veloce del tempo reale: nel file SDF il target è `real_time_factor=1.0`. L'esecuzione headless e la GUI separata riducono il carico grafico e possono migliorare la fluidità, ma il fattore ottenuto va misurato a runtime.

Documento basato sullo stato dei file presenti nel workspace al 4 ottobre 2026. Descrive comportamenti configurati nel codice; non certifica l'esecuzione o il collaudo su hardware.

Struttura del progetto

Il repository separa l'orchestrazione Docker, l'immagine degli strumenti ROS, gli asset di simulazione e il workspace colcon. Le mesh e le texture restano file di risorsa: non sono codice ROS e vengono montate nel container del simulatore.

```

ASP_project/
├── docker-compose.yml          # servizi sim, dev e agent
├── docker/
│   └── ros/
│       ├── Dockerfile          # immagine ROS 2 + dipendenze PX4/Gazebo
│       └── entrypoint.sh        # setup degli overlay ROS
├── gazebo/
│   ├── worlds/warehouse_px4.sdf # fisica e layout del magazzino
│   ├── models/                  # modelli, SDF, mesh e materiali
│   └── x500/                     # modello Gazebo x500 disponibile localmente
├── ros2_ws/
│   ├── scripts/                 # spazio per script del workspace
│   └── src/warehouse_bringup/
│       ├── launch/sim_bringup.launch.py
│       ├── warehouse_bringup/takeoff_1m.py
│       ├── package.xml
│       ├── setup.py / setup.cfg
│       └── resource/warehouse_bringup
├── scripts/start.sh             # avvio, arresto, stato e GUI
└── docs/relazione_progetto.html # sorgente modificabile di questo PDF

```

Immagine ROS

`docker/ros/Dockerfile` parte da `ros:humble-ros-base`, installa gli strumenti di build e clona Micro XRCE-DDS Agent e `px4_msgs`. Con `WITH_GZ=1` (default) aggiunge Gazebo Harmonic e `ros_gz_bridge`. I riferimenti dell'agent e dei messaggi sono controllabili dagli ARG.

Package ROS 2

`warehouse_bringup` è un package Python ament. Le dipendenze runtime includono `launch/launch_ros`, `ros_gz_bridge`, `rclpy` e `px4_msgs`. `setup.py` espone l'eseguibile ROS `takeoff_1m`.

Overlay. L'entrypoint carica in sequenza ROS Humble e `/deps_ws/install`; se esiste, carica anche `/ws/install`. Il bind mount `./ros2_ws/src:/ws/src` rende visibili nel container le modifiche locali ai package.

PX4, Gazebo e warehouse

Il servizio `sim` usa l'immagine `px4io/px4-sitl-gazebo:latest`, seleziona `gz_x500_depth` e carica il world `warehouse_px4`. La posa iniziale predefinita è `-19,35,0,0,0,0` ed è sovrascrivibile tramite la variabile d'ambiente `PX4_GZ_MODEL_POSE`.

Parametro	Configurazione	Effetto pratico
Fisica SDF	<code>ode, max_step_size=0.004</code>	Passo temporale di 4 ms, equivalente a 250 step/s.
Velocità simulata	<code>real_time_factor=1.0</code>	Target pari al tempo reale, non un'accelerazione temporale.
Aggiornamento	<code>real_time_update_rate=250</code>	Coerente con 1 / 0,004 s per passo.
Rendering server	<code>HEADLESS=1</code>	Il server non avvia la GUI; riduce il lavoro grafico sul processo di simulazione.
Partizione Gazebo	<code>GZ_PARTITION=warehouse</code>	Isola e allinea i nodi Gazebo che comunicano tra host e container.
Rete e GPU	<code>network host; /dev/dri</code> esposto	Comunicazione diretta sull'host e disponibilità del device grafico al container.

Composizione del mondo

Il file `warehouse_px4.sdf` contiene il piano base e 71 istanze incluse, disposte tramite pose esplicite. Conteggio delle famiglie presenti nel world:

Famiglia	Istanze	Uso
ShelfF	32	Scaffalature principali, distribuite in file e posizioni nominate.
Cluttering A / C / D	12 ciascuna	Ostacoli e oggetti di ingombro tra gli scaffali.
ShelfD	2	Elementi aggiuntivi di scaffalatura.
GroundB	1	Elemento warehouse a pavimento.

Nel conteggio SDF ci sono 72 dichiarazioni `<model>`: il piano base più 71 riferimenti `model://`. `Bucket`, `ShelfE` e `TrashCanC` sono anche montati nel container, ma non risultano inclusi nel world attuale. PX4 aggiunge il drone all'avvio.

Ogni cartella modello contiene tipicamente `model.config`, `model.sdf` e mesh visuali/collisioni in formato DAE; alcune famiglie includono materiali e texture. Le directory sono montate in sola lettura sotto `/opt/px4-gazebo/share/gz/models`.

Bridge e avvio ROS 2

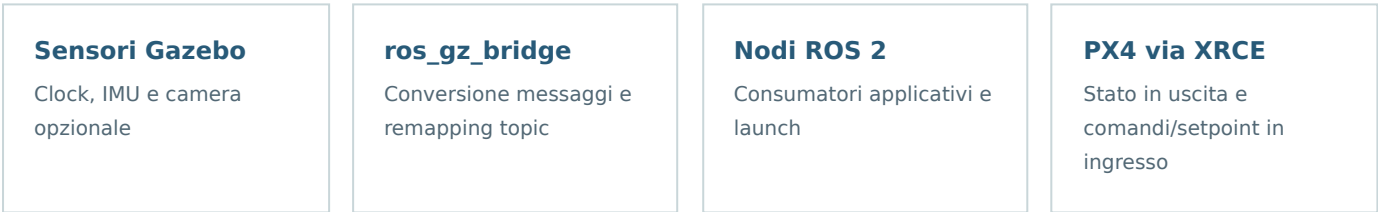
Il servizio `agent` esegue `MicroXRCEAgent udp4 -p 8888`. Con rete host, PX4 SITL e i nodi ROS possono scambiare i messaggi uORB esposti da `px4_msgs` tramite il trasporto DDS-XRCE. Il servizio `dev` offre la shell interattiva ROS 2 sullo stesso network namespace.

Launch configurabile

`sim_bringup.launch.py` abilita opzionalmente bridge, camera e nodo takeoff. Valori predefiniti: `sim=true`, `world=warehouse_px4`, `model=x500_depth_0`, `takeoff=false`, `camera=false`.

Origine Gazebo	Topic ROS	Messaggio ROS	Abilitazione
<code>/clock</code>	<code>/clock</code>	<code>rosgraph_msgs/msg/Clock</code>	Quando <code>sim=true</code>
sensore IMU del link drone	<code>/imu/data</code>	<code>sensor_msgs/msg/Imu</code>	Quando <code>sim=true</code>
sensore IMX214: <code>image</code>	<code>/camera/image_raw</code>	<code>sensor_msgs/msg/Image</code>	<code>sim=true</code> e <code>camera=true</code>
sensore IMX214: <code>camera_info</code>	<code>/camera/camera_info</code>	<code>sensor_msgs/msg/CameraInfo</code>	<code>sim=true</code> e <code>camera=true</code>

Flusso dei dati



Se `sim=false`, il launch non crea il bridge Gazebo. Questa opzione, da sola, non configura un driver o una connessione a un drone reale: il package è predisposto a separare quel bridge, ma l'integrazione hardware non è descritta né verificata dai file esaminati.

Il bridge camera è disattivato per default: riduce traffico e consumo quando non serve elaborazione visiva. L'attivazione resta esplicita con `camera:=true`.

Takeoff Offboard a un metro

- Il nodo `takeoff_1m` è un esempio di controllo posizione tramite interfaccia Offboard PX4. Non parte automaticamente: il launch dichiara `takeoff=false` per default, quindi va richiesto consapevolmente.
- 01 Attende una stima locale valida.** Si sottoscrive a `/fmu/out/vehicle_local_position_v1` con QoS best-effort e acquisisce una sola volta la posizione iniziale quando `xy_valid` e `z_valid` sono veri.
 - 02 Pubblica heartbeat e setpoint.** Un timer ogni 0,1 s (10 Hz) invia `OffboardControlMode` con controllo posizione e `TrajectorySetpoint` verso `/fmu/in/...`.
 - 03 Richiede Offboard.** Dopo almeno 10 pubblicazioni preliminari, invia `VEHICLE_CMD_DO_SET_MODE` con i parametri di modalità Offboard. Le richieste vengono ripetute al massimo una volta al secondo finché lo stato conferma la modalità.
 - 04 Richiede l'armamento.** Quando PX4 segnala Offboard attivo, invia `VEHICLE_CMD_COMPONENT_ARM_DISARM` con armamento abilitato, ripetendo con intervallo di un secondo finché riceve conferma.
 - 05 Mantiene la quota.** Continua a inviare lo stesso setpoint e registra una sola volta che il drone è armato e il target viene mantenuto.

Convenzione verticale. PX4 usa coordinate locali NED: l'asse Z cresce verso il basso. Per salire di 1 m il codice imposta `z_target = z_start - 1.0`, lasciando X e Y sulla posizione iniziale.

Topic e QoS

Direzione	Topic	Scopo
ROS → PX4	<code>/fmu/in/offboard_control_mode</code>	Indica il tipo di setpoint e mantiene attiva la modalità esterna.
ROS → PX4	<code>/fmu/in/trajectory_setpoint</code>	Target posizione; velocità, accelerazione, jerk e yaw sono NaN.
ROS → PX4	<code>/fmu/in/vehicle_command</code>	Richieste di cambio modalità e armamento.
PX4 → ROS	<code>/fmu/out/vehicle_local_position_v1</code>	Posizione locale e flag di validità.
PX4 → ROS	<code>/fmu/out/vehicle_status_v4</code>	Conferma stato Offboard e armamento.

Il timer usa il clock del nodo e i timestamp dei messaggi sono espressi in microsecondi. Le richieste sono considerate riuscite quando cambia lo stato osservato, non sulla sola pubblicazione del comando.

Limite di sicurezza. Questo nodo è dimostrativo: non implementa atterraggio, disarmo, timeout di perdita posizione, controllo ostacoli o gestione esplicita dei rifiuti PX4. Non sostituisce una procedura operativa validata su velivolo reale.

Avvio, prestazioni e verifiche

Script di orchestrazione

`scripts/start.sh` offre i comandi `start`, `stop`, `restart` e `status`, oltre a `--no-gui` e `--no-shell`. Prima dell'avvio controlla Docker Compose e la presenza del world. Avvia in background `sim` e `agent`, poi attende i topic del world e lo spawn del modello. La GUI Gazebo, quando richiesta e disponibile sull'host, viene avviata separatamente con `gz sim -g`. Infine apre una shell nel servizio `dev`.

```
scripts/start.sh start
scripts/start.sh start --no-gui
scripts/start.sh status
scripts/start.sh stop
```

Dentro la shell ROS, una sequenza tipica per compilare e avviare il bringup è:

```
source /opt/ros/humble/setup.bash
colcon build --symlink-install
source install/setup.bash
ros2 launch warehouse_bringup sim_bringup.launch.py
```

Per attivare il decollo di esempio e il bridge camera, aggiungere `takeoff:=true camera:=true` al comando di `launch`. L'avvio grafico dell'host è controllabile in modo indipendente dal server headless.

In che senso è stata “accelerata”

Intervento presente	Che cosa migliora	Che cosa non garantisce
Gazebo server con <code>HEADLESS=1</code>	Evita rendering e GUI nel container; riduce contesa CPU/GPU e overhead grafico.	Non cambia da solo la velocità del tempo simulato.
GUI sul PC host, avviabile separatamente	Il rendering non è accoppiato al ciclo di vita del processo server; <code>--no-gui</code> elimina anche quel carico.	Non imposta un RTF superiore a 1.
Passo 4 ms e update rate 250 Hz	Fissa una discretizzazione coerente per la dinamica configurata.	È una frequenza di simulazione target, non un moltiplicatore di velocità.

Conclusione verificabile: il progetto è ottimizzato per alleggerire l'esecuzione e può quindi raggiungere più facilmente il tempo reale; non contiene al momento una configurazione di accelerazione temporale oltre il tempo reale. `status` legge il valore runtime di `real_time_factor` dal topic `/stats`, che è il controllo da usare dopo l'avvio.

Stato e prossimi collaudi utili

Nel workspace sono presenti l'orchestrazione, i modelli warehouse, il world, la definizione del package, il bridge configurabile e il nodo di takeoff. Per una verifica completa restano da eseguire build dell'immagine, avvio in ambiente, compilazione colcon, controllo topic/stati PX4 e misura del RTF sotto carico. Non è stato possibile ricostruire una cronologia Git: la cartella di lavoro non risulta un repository Git.

Riferimenti principali nel progetto: `docker-compose.yml`, `docker/ros/Dockerfile`, `docker/ros/entrypoint.sh`, `scripts/start.sh`, `gazebo/worlds/warehouse_px4.sdf` e i file del package `ros2_ws/src/warehouse_bringup`.